

Homework 2 — Conditionals

Homework 2 — Conditionals

Due Sunday 4 October, 23:59, on Canvas. The deadline is fixed; late work is not received.

Individual work. Two programs. Same rules as Homework 1, plus one: **the tests are part of the submission** — write, under each algorithm, the inputs you tried and what the program printed, and make sure they include the boundaries.

What you submit

For **each** program, three things:

1. **The handwritten algorithm** — photo or scan, or the `A_Matricula.docx` template. INPUT: line, OUTPUT: line, numbered steps, with every IF / ELSE written out.
2. **The .py file**, named as stated, in the exam format: line 1 your name in a # comment, then the algorithm inside a `"""` multi-line comment, then the code.
3. **Your tests** — the table of inputs and outputs you checked, including the ones at the edges.

A submission with code but no algorithm is **incomplete** and is graded as such. See *Using AI in this course*.

Problem 1 — Leap year (`A_Matricula_hw2a.py`)

A year is a leap year when it is divisible by 4 **and** not divisible by 100, **or** when it is divisible by 400. Read a year from the user and display whether it is a leap year.

Required tests, with what your program printed for each:

year	expected
1984	leap year
1800	not a leap year
2000	leap year
2026	not a leap year
2100	not a leap year

Then answer, in one sentence under the algorithm: *why is 2100 not a leap year although it is divisible by 4?* Point at the part of your condition that says so.

Problem 2 — Three numbers, descending (*A_Matricula_hw2b.py*)

Read three numbers from the user and display them from largest to smallest.

Example: -9, 4, 1 → 4 1 -9.

No lists, no `sorted()` — we have not met them, and the point is the decisions. Write the algorithm first: you will find you need to compare pairs, and that the order in which you compare them matters. Say, in the algorithm, what happens when two numbers are equal.

Required tests:

input	expected
-9 4 1	4 1 -9
1 2 3	3 2 1
3 2 1	3 2 1
5 5 2	5 5 2
7 7 7	7 7 7

Rubric — published

Algorithm present, with input/output and every branch stated	40 %
Program runs and passes all required tests	40 %
Boundaries and equal values handled on purpose (2100, 5 5 2, 7 7 7)	20 %

Two points off for each missing technical requirement (name comment, algorithm in `"""`, file name, tests written down). The two programs weigh the same.